

Vision Meets Language

Building a Multimodal Transformer from Scratch

Seasons of Code · IIT Bombay

Mentee Handbook · Tasks 3 and 4

Vision Enters the Picture

Preface to Tasks 3 and 4

Where we are. If you completed Task 2, you have built a working decoder-only transformer in PyTorch from scratch. You have derived the gradient of attention by hand. You understand multi-head attention, residual connections, LayerNorm, and the role of the MLP. That is a substantial foundation, and from here on, everything is a variation on what you already know.

What's coming. Task 3 introduces vision. For most of you this is the first time treating images as input to a neural network. We start genuinely from the basics — what is an image to a computer, how does a convolution work, why do CNNs dominate vision — before reaching the Vision Transformer (ViT). The key insight you'll arrive at is that ViT *is the same transformer you already built*, with one change: instead of feeding it word tokens, you feed it image patches.

Task 4 bridges the two worlds. Cross-attention is a small mechanical change from the self-attention you wrote in Task 1. The queries come from one modality (say, text), and the keys and values come from another (say, image patch embeddings). That tiny change is the entire mechanism by which modern multimodal models like CLIP, Flamingo, and BLIP let vision and language talk to each other.

Pace. These two tasks together will take roughly 9–10 days. Vision is new ground for most of you, so we have budgeted slightly more time per day (5–6 hours instead of the 4–5 of earlier tasks). If you find Task 3's CNN day easy, push ahead. If patching feels confusing, slow down — it is a small idea but trips many people up the first time.

As always — writeups matter more than code. Every task has conceptual questions at the end. These are the actual signal of understanding.

Task 3 — From Pixels to Patches: Vision Transformers from Scratch

Time budget: 28–32 hours over 5–6 days.

Goal: Understand how images become tensors, build a tiny CNN to feel why convolutions are natural for images, then build a Vision Transformer from scratch and train it on a real image classification dataset.

Why this task exists

The original ViT paper (Dosovitskiy et al., 2020) is titled "An Image is Worth 16x16 Words" because that is exactly the core idea: cut an image into a grid of patches, flatten each patch into a vector, and now you have a sequence of "image tokens" that you can feed to the same transformer you already built. The radical claim of that paper — surprising in 2020 — was that you don't need convolutions at all if you have enough data.

To appreciate why that was surprising, you first need to know what convolutions do and why they had been the only serious answer to image processing for the previous decade. That's why this task starts with a one-day CNN detour before we get to ViT.

Part A — Images as Tensors

Time: about 3 hours.

In a single notebook, `task3/images_as_tensors.ipynb`, explore how PyTorch represents images.

1. Install torchvision: `pip install torchvision`. Download MNIST (handwritten digits, 28×28 grayscale) with `torchvision.datasets.MNIST`.
2. Print the shape of a single image. Note: PyTorch uses the convention (C, H, W) — channels first. For MNIST, C=1 (grayscale). For natural images, C=3 (RGB).
3. Visualize the first 16 images using matplotlib. Use `plt.imshow(img.squeeze(), cmap='gray')` — the squeeze removes the channel dimension for display.
4. Print pixel statistics: min, max, mean. MNIST pixels are normalized to [0, 1]. Real photographs usually need standardization (subtract dataset mean, divide by std) before training.
5. Download CIFAR-10 (32×32 color images, 10 classes). Print the shape of one image. Visualize 16 images with their class labels.
6. Build a DataLoader with batch size 64. Iterate one batch and print the shapes of x (the image batch) and y (the labels).



CURIOSITY CORNER — Channel conventions and why they matter

PyTorch uses (C, H, W). TensorFlow defaults to (H, W, C). NumPy and most image libraries (PIL, OpenCV) use (H, W, C). This trips up nearly everyone the first time. When you load an image with PIL and convert to a tensor, the conversion handles the swap for you — but if you ever build tensors manually from numpy arrays, watch out.

Also: OpenCV reads images in BGR order, not RGB. This has caused more bugs in computer vision projects than almost anything else. If your model trained fine but predictions on new images look weird, check your channel order first.

CURIOSITY CORNER — Data augmentation — the unsung hero of computer vision

Real vision models almost always train with random augmentations: horizontal flips, random crops, color jitter, rotations. Each training image gets a slightly different version every epoch. This is essentially free regularization — it teaches the model that a cat is still a cat after a small shift or color change, without you needing to collect more data.

`torchvision.transforms` gives you a clean API for chaining augmentations. For ViT specifically, modern recipes use heavy augmentation (RandAugment, Mixup, CutMix) because transformers are data-hungry and need this to avoid overfitting on smaller datasets.

Part B — Build a Tiny CNN (the Vision Baseline)

Time: about 5 hours.

Before you write a ViT, build a small CNN on CIFAR-10. The goal is not to get great accuracy — it is to feel why convolutions are a natural fit for images, so you appreciate what ViT replaces. Save as `task3/cnn_baseline.py`.

Architecture (build this exact one):

```
class TinyCNN(nn.Module):
    def __init__(self, num_classes=10):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 32, kernel_size=3, padding=1)
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
        self.conv3 = nn.Conv2d(64, 128, kernel_size=3, padding=1)
        self.pool = nn.MaxPool2d(2, 2)
        self.fc1 = nn.Linear(128 * 4 * 4, 256)
        self.fc2 = nn.Linear(256, num_classes)
        self.dropout = nn.Dropout(0.3)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x))) # 32x32 -> 16x16
        x = self.pool(F.relu(self.conv2(x))) # 16x16 -> 8x8
        x = self.pool(F.relu(self.conv3(x))) # 8x8 -> 4x4
```

```
x = x.flatten(1) # (B, 128*4*4)
x = self.dropout(F.relu(self.fc1(x)))
return self.fc2(x)
```

Training: CIFAR-10, AdamW, learning rate 1e-3, 10 epochs, batch size 64. Track training and validation accuracy. You should reach around 65–70% validation accuracy.

Manual exercise (do this on paper before running): for one Conv2d layer, given input shape (B, 3, 32, 32), kernel size 3, padding 1, stride 1, compute the output shape. Then verify with PyTorch by printing intermediate shapes in forward.

CURIOSITY CORNER — What a convolution actually computes

A 2D convolution slides a small learnable kernel (say 3×3) across the image and computes a dot product at every position. The same kernel weights are used at every spatial location — this is the famous "weight sharing" of CNNs, and it's why they have far fewer parameters than fully-connected networks on images.

Three deep properties fall out of this: (1) translation equivariance — if you shift the input, the output shifts the same way; (2) locality — each output position only depends on a small neighborhood of the input; (3) hierarchical features — stacking convolutions lets each layer combine features from a larger receptive field. These three properties together are why CNNs dominated vision for a decade.

CURIOSITY CORNER — The ImageNet moment and the decade of CNNs

In 2012, AlexNet won the ImageNet competition by a huge margin using deep convolutional networks on GPUs. This is widely considered the start of the modern deep learning era. From 2012 to 2020, essentially every state-of-the-art vision model was a CNN: VGG (2014), ResNet (2015), Inception, DenseNet, EfficientNet.

ResNet in particular introduced residual connections — yes, the same residuals you used in Task 2 — to enable training networks 100+ layers deep. The transformer architecture (2017) borrowed residuals directly from ResNet. The lineage matters: many "transformer" tricks are CNN tricks in disguise.

Part C — The Patch Embedding

Time: about 4 hours.

This is the conceptual heart of ViT. Take a moment with it.

The core idea. An image of shape (3, 32, 32) has 3072 pixels. You could flatten it into a sequence of 3072 tokens and feed it to a transformer — but that's far too long (attention is $O(T^2)$, so this would be 9

million attention scores). Instead, ViT cuts the image into a grid of non-overlapping patches — say 4×4 patches, giving a 8×8 grid = 64 patches. Each patch contains 4×4×3 = 48 pixels. Flatten each patch to a vector of length 48, then linearly project to your model dimension. Now you have a sequence of 64 "patch tokens" of dimension `n_embd`. Feed to transformer.

Two ways to implement patching:

Method 1 — Using `einops` or `reshape`.

```
# x: (B, 3, 32, 32), patch_size=4
# Goal: (B, 64, 48) where 64 = (32/4)*(32/4), 48 = 4*4*3
patches = x.unfold(2, 4, 4).unfold(3, 4, 4)
# patches: (B, 3, 8, 8, 4, 4)
patches = patches.permute(0, 2, 3, 1, 4, 5).flatten(1, 2).flatten(2)
# patches: (B, 64, 48)
```

Method 2 — Using `Conv2d` with a clever trick (this is what real ViT code does).

```
# A Conv2d with kernel_size=patch_size and stride=patch_size
# acts as a non-overlapping patch projector!
self.patch_embed = nn.Conv2d(
    in_channels=3,
    out_channels=n_embd,
    kernel_size=patch_size,
    stride=patch_size,
)
# x: (B, 3, 32, 32) -> conv -> (B, n_embd, 8, 8)
# flatten spatial dims and transpose -> (B, 64, n_embd)
x = self.patch_embed(x).flatten(2).transpose(1, 2)
```

Implement both methods. Verify they produce the same result up to the linear projection weights.

Method 2 is what you'll use in your ViT — it folds patch extraction and the linear projection into a single efficient operation.

CURIOSITY CORNER — Why patches and not pixels?

Self-attention is $O(T^2)$ in sequence length. For a 224×224 image, pixels would mean $T = 50,176$ tokens, and attention would need 2.5 billion pairwise scores. Patches reduce sequence length by patch_size^2 — with 16×16 patches, you get $T = 196$, which is comparable to a sentence.

There's also a representational argument: individual pixels carry almost no information. Patches encode small textures, edges, and color blobs that are meaningful units to attend over. Patches are to images roughly what subword tokens are to text — small chunks of structure that the model learns to compose.

CURIOSITY CORNER — The CLS token — borrowed from BERT

ViT prepends a special learnable [CLS] token to the sequence of patch tokens. After all the transformer blocks, the model's classification head reads from the final state of this CLS token only — not from the patch tokens. The idea (borrowed directly from BERT in NLP) is that the CLS token serves as a learned "summary" slot that aggregates information from all patches via attention.

Modern variants sometimes use global average pooling over patch tokens instead, and the difference is small. But the CLS token is in the original paper and you should implement it.

Part D — Position Embeddings for Images

Time: about 2 hours.

Transformers have no built-in notion of order. In Task 1 you added learned position embeddings to text tokens. ViT does the same — adds a learned position embedding to each patch token. The position embedding tells the model "this patch came from the top-left" versus "this patch came from the middle".

Implementation:

```
# num_patches = (img_size // patch_size) ** 2
# +1 for the CLS token
self.pos_embed = nn.Parameter(
    torch.zeros(1, num_patches + 1, n_embd)
)
nn.init.trunc_normal_(self.pos_embed, std=0.02)
```

Then in forward, after constructing the patch token sequence with CLS prepended:

```
x = x + self.pos_embed # broadcasts over batch
```

CURIOSITY CORNER — Learned vs sinusoidal vs 2D position embeddings

The original Transformer used fixed sinusoidal position embeddings — no learning, just sines and cosines at different frequencies. ViT uses learned 1D position embeddings (each patch position has its own learnable vector). 2D position embeddings (separate row and column embeddings) are also a choice. Surprisingly, the original ViT paper found that 1D, 2D, and even no position embeddings all gave similar results when the model was large enough — which is wild.

More modern approaches: RoPE (Rotary Position Embedding) rotates query and key vectors based on their position, used in LLaMA and many recent LLMs. ALiBi adds a position-dependent bias directly to attention scores. For our project, learned 1D positions are fine.

Part E — Assemble the Vision Transformer

Time: about 6 hours.

Now bring it all together in `task3/vit.py`. The transformer Block class from Task 2 is reused as-is, with one change: **remove the causal mask**. In ViT, every patch can attend to every other patch — there is no temporal ordering to respect. Patches in row 5 should freely look at patches in row 2.

ViT architecture:

```
class ViT(nn.Module):
    def __init__(self, img_size=32, patch_size=4, in_chans=3,
                  num_classes=10, n_embd=192, n_head=6,
                  n_layer=6, dropout=0.1):
        super().__init__()
        num_patches = (img_size // patch_size) ** 2

        # Patch embedding (conv trick)
        self.patch_embed = nn.Conv2d(
            in_chans, n_embd,
            kernel_size=patch_size, stride=patch_size
        )

        # CLS token and position embedding
        self.cls_token = nn.Parameter(torch.zeros(1, 1, n_embd))
        self.pos_embed = nn.Parameter(
            torch.zeros(1, num_patches + 1, n_embd)
        )
        self.dropout = nn.Dropout(dropout)

        # Transformer blocks (your Task 2 Block, but no causal mask)
        self.blocks = nn.ModuleList([
            BlockBidirectional(n_embd, n_head, dropout)
            for _ in range(n_layer)
        ])
        self.norm = nn.LayerNorm(n_embd)

        # Classification head (reads from CLS only)
        self.head = nn.Linear(n_embd, num_classes)

        # Init
        nn.init.trunc_normal_(self.cls_token, std=0.02)
        nn.init.trunc_normal_(self.pos_embed, std=0.02)

    def forward(self, x):
        B = x.size(0)

        # Patchify
        x = self.patch_embed(x)           # (B, C, H/p, W/p)
        x = x.flatten(2).transpose(1, 2)  # (B, num_patches, C)

        # Prepend CLS token
        cls = self.cls_token.expand(B, -1, -1)
```

```

x = torch.cat([cls, x], dim=1)          # (B, 1+num_patches, C)

# Add position embedding
x = x + self.pos_embed
x = self.dropout(x)

# Transformer blocks
for blk in self.blocks:
    x = blk(x)
x = self.norm(x)

# Classify from CLS token
cls_final = x[:, 0]                    # (B, C)
return self.head(cls_final)

```

⚠ Remove the causal mask in your Block

Your Task 2 Block has a causal mask baked into the Head class via the registered tril buffer. For ViT you need bidirectional attention — every patch sees every other patch. Either create a new BlockBidirectional class without the mask, or add a flag to your Head class to toggle the mask. Forgetting this is the single most common bug at this stage.

Training. Train on CIFAR-10. Hyperparameters: patch_size=4, n_embd=192, n_head=6, n_layer=6, lr=3e-4, AdamW, 30 epochs, batch size 128, dropout 0.1.

Expected outcome: around 65–72% validation accuracy on CIFAR-10. This is similar to your CNN baseline, sometimes slightly worse — and that's an important lesson. ViT shines on large datasets; on tiny CIFAR-10, the CNN's inductive biases (locality, translation equivariance) actually help. ViT needs more data or heavier augmentation to beat CNNs.

🔍 CURIOSITY CORNER — Why ViT struggles on small datasets

CNNs have inductive biases hard-coded into their architecture: translation equivariance (a cat is a cat regardless of where in the image it sits) and locality (nearby pixels are more related than distant ones). These biases mean CNNs can generalize well from limited data — the architecture itself encodes useful prior knowledge.

ViT has none of this. To a ViT, the patch at position (0, 0) and the patch at position (7, 7) are just two tokens in a sequence — it has to learn from scratch that nearby patches are usually related. With enough data (ImageNet-21k, JFT-300M), it learns this beautifully and surpasses CNNs. With CIFAR-10's 50k images, it's working against itself.

This trade-off — inductive bias vs scale — is one of the most important themes in modern ML. Less bias requires more data, but also lets the model discover patterns the bias would have hidden.

CURIOSITY CORNER — Attention maps — what is your ViT actually looking at?

Once trained, you can extract the attention weights from the final block's CLS token (its attention over all patches). Reshape these back to a 2D grid and overlay on the original image — you get a heatmap showing what the model attends to when classifying. For a well-trained ViT on a cat image, attention focuses on the cat. This is a popular Task 8 extension and very satisfying to visualize.

Caveat: raw attention weights are not a perfect explanation. The model can route information through residuals in ways attention maps don't show. Search "attention rollout" and "attention is not explanation" for the debate.

Part F — Conceptual Writeup

Time: 2–3 hours.

Answer in `task3/writeup.md`:

7. Compare your CNN baseline and your ViT on CIFAR-10. Which got better validation accuracy? Why might that be — what advantages does each architecture have?
8. In your own words, explain why patching is necessary for ViT. Why not feed pixels directly?
9. Explain the role of the CLS token. Why does the classifier read from CLS rather than averaging over patch tokens?
10. In Task 2 you used a causal mask. In Task 3 (ViT) you removed it. Explain why this difference makes sense for the two tasks. What would happen to your ViT if you accidentally kept the causal mask?
11. Position embeddings for text encode token order. What do position embeddings for image patches encode, and why does the model need them?
12. What did you find hardest? What clicked unexpectedly?

Deliverables for Task 3

- `task3/images_as_tensors.ipynb`
- `task3/cnn_baseline.py` with training and validation curves saved as a plot.
- `task3/vit.py` with training and validation curves saved as a plot.
- `task3/comparison_plot.png` — CNN and ViT validation accuracy on the same axes.
- `task3/writeup.md`

Task 4 — Cross-Attention: The Multimodal Bridge

Time budget: 18–22 hours over 4 days.

Goal: Implement cross-attention from scratch as a small variation on self-attention, then use it to wire together your Task 2 text transformer and your Task 3 ViT so that text tokens can attend to image patches.

Why this task exists

Every multimodal model in the modern era — CLIP, Flamingo, BLIP, LLaVA, GPT-4V — relies on attention to connect modalities. The mechanism is almost trivially simple: queries come from one stream, keys and values come from another. That's it. The conceptual leap from self-attention to cross-attention is a one-line change. But that one line is the entire foundation of multimodal AI, so we take a full task to make sure you internalize it and have working code that uses it.

Part A — From Self-Attention to Cross-Attention

Time: about 3 hours.

The one-line change. In self-attention, Q, K, and V all come from the same input x:

```
# Self-attention
k = self.key(x)
q = self.query(x)
v = self.value(x)
```

In cross-attention, Q comes from one source and K, V come from another:

```
# Cross-attention: query from "x" (e.g. text)
#                      keys/values from "context" (e.g. image)
q = self.query(x)      # x: (B, T_x, C)
k = self.key(context)   # context: (B, T_c, C)
v = self.value(context) # context: (B, T_c, C)
```

Resulting shapes. Attention scores are now (B, T_x, T_c) — every position in x can attend to every position in context. The output has shape $(B, T_x, \text{head_size})$ — same length as the query sequence. Cross-attention *transforms* the query sequence using information pulled from the context.

Build a CrossAttentionHead class:

```
class CrossAttentionHead(nn.Module):
    def __init__(self, n_embd, head_size):
        super().__init__()
        self.key = nn.Linear(n_embd, head_size, bias=False)
        self.query = nn.Linear(n_embd, head_size, bias=False)
        self.value = nn.Linear(n_embd, head_size, bias=False)

    def forward(self, x, context):
        # x: (B, T_x, C) - queries source
```

```

# context: (B, T_c, C) - keys/values source
q = self.query(x)
k = self.key(context)
v = self.value(context)

wei = q @ k.transpose(-2, -1)
wei = wei * (k.size(-1) ** -0.5)
wei = F.softmax(wei, dim=-1)
# No causal mask in cross-attention by default
return wei @ v

```

Then wrap into a **MultiHeadCrossAttention** class following the same pattern as your Task 2 multi-head self-attention.

CURIOSITY CORNER — Where does the causal mask go in cross-attention?

In encoder-decoder architectures (the original Transformer for translation, for instance), cross-attention in the decoder is NOT causally masked over the context — the decoder can see all encoder outputs at once. But the decoder's self-attention IS causally masked, because the decoder is autoregressive.

So in a decoder block that uses cross-attention, you typically have three sub-layers: (1) causal self-attention on the decoder sequence, (2) cross-attention onto the encoder output (no mask), (3) MLP. We'll see this exact pattern in Part C.

CURIOSITY CORNER — Cross-attention has been around longer than you think

The original 2017 Transformer was an encoder-decoder for machine translation. The decoder cross-attended onto the encoder's output. So cross-attention is in the very first transformer paper — it's not a multimodal-specific invention.

What multimodal models did was realize that the "encoder" doesn't have to be text. It can be image features, audio, video — anything. The attention mechanism doesn't care about modality; it just needs vectors of the right dimension. That's the deep reason transformers came to dominate every domain.

Part B — Cross-Attention on Toy Data

Time: about 4 hours.

Before wiring up real models, sanity-check your cross-attention on toy tensors. Save as `task4/cross_attention_toy.ipynb`.

13. Create dummy input: `x` of shape `(2, 10, 64)` (batch 2, query length 10, dim 64) and context of shape `(2, 5, 64)` (context length 5).

14. Instantiate your MultiHeadCrossAttention with `n_embd=64`, `n_head=4`, `head_size=16`. Pass `x` and `context` through it. Verify output shape is `(2, 10, 64)` — same as query input.
15. Extract the attention weights from one head and visualize them as a heatmap. Rows are query positions (0–9), columns are context positions (0–4). Each row should sum to 1.
16. Construct a structured test: make `context[0, 2]` identical to `x[0, 5]` (same vector). After untrained random init, the attention from query position 5 should not particularly favor context position 2 — confirm this. Then train a tiny loop where the loss encourages the output at position 5 to match `context[0, 2]` specifically. After 1000 steps of optimization, re-extract the attention weights and verify position 5 now strongly attends to position 2.

Why this matters. This is your sanity check that cross-attention can learn to route information from specific context positions to specific query positions. If this toy experiment doesn't work, real cross-attention won't either.

CURIOSITY CORNER — Attention as a soft database lookup

A useful mental model: cross-attention is a learned, differentiable, soft version of a key-value database lookup. The query is your search request. Each context position has a key (its searchable label) and a value (its stored content). The softmax over q-k similarities gives you a soft distribution over which keys best match, and you retrieve a weighted average of the corresponding values.

The reason this is powerful: every step is differentiable. You can train the keys, values, and queries end-to-end so the network discovers what to look up and what to store. This is the deep idea behind retrieval-augmented generation (RAG), memory networks, and many other architectures.

Part C — Wire Up Your Models: Text Cross-Attends to ViT

Time: about 8 hours.

Now build a real multimodal model in `task4/multimodal.py` that combines your Task 3 ViT (vision encoder) with your Task 2 transformer (now a text decoder with cross-attention added).

Architecture:

- **Vision encoder:** your ViT from Task 3, but without the classification head. Take the patch token outputs (or all tokens including CLS) as the image representation. Shape: `(B, num_patches+1, n_embd_img)`.
- **Text decoder:** your Task 2 transformer, modified. Each block now has three sub-layers instead of two: causal self-attention, then cross-attention onto the image features, then MLP.
- **Output:** the same language modeling head — predict the next character.

The modified decoder block:

```

class DecoderBlock(nn.Module):
    def __init__(self, n_embd, n_head, block_size, dropout=0.1):
        super().__init__()
        head_size = n_embd // n_head
        self.self_attn = MultiHeadAttention(
            n_embd, n_head, head_size, block_size
        ) # causal
        self.cross_attn = MultiHeadCrossAttention(
            n_embd, n_head, head_size
        ) # not causal
        self.ffn = FeedForward(n_embd)
        self.ln1 = nn.LayerNorm(n_embd)
        self.ln2 = nn.LayerNorm(n_embd)
        self.ln3 = nn.LayerNorm(n_embd)

    def forward(self, x, image_features):
        x = x + self.self_attn(self.ln1(x))
        x = x + self.cross_attn(self.ln2(x), image_features)
        x = x + self.ffn(self.ln3(x))
        return x

```

Important detail — matching dimensions. Your ViT might output features with a different `n_embd` than your text model. Add a linear projection to align them, or set the two `n_embd` values to be equal from the start. For this task, use the same `n_embd` for both (e.g., 128) to keep things simple.

Full multimodal model:

```

class MultimodalModel(nn.Module):
    def __init__(self, vocab_size, n_embd=128, n_head=4,
                 n_layer=4, block_size=64, img_size=32,
                 patch_size=4):
        super().__init__()
        self.vision_encoder = ViTEncoder(
            img_size=img_size, patch_size=patch_size,
            n_embd=n_embd, n_head=n_head, n_layer=n_layer
        )
        self.token_embed = nn.Embedding(vocab_size, n_embd)
        self.pos_embed = nn.Embedding(block_size, n_embd)
        self.blocks = nn.ModuleList([
            DecoderBlock(n_embd, n_head, block_size)
            for _ in range(n_layer)
        ])
        self.ln_final = nn.LayerNorm(n_embd)
        self.lm_head = nn.Linear(n_embd, vocab_size)
        self.block_size = block_size

    def forward(self, image, text_ids, targets=None):
        # Encode image
        image_feats = self.vision_encoder(image)
        # (B, num_patches+1, n_embd)

```

```

# Embed text
B, T = text_ids.shape
tok = self.token_embed(text_ids)
pos = self.pos_embed(torch.arange(T, device=text_ids.device))
x = tok + pos

# Decoder blocks with cross-attention to image
for blk in self.blocks:
    x = blk(x, image_feats)
x = self.ln_final(x)
logits = self.lm_head(x)

loss = None
if targets is not None:
    loss = F.cross_entropy(
        logits.view(-1, logits.size(-1)),
        targets.view(-1)
    )
return logits, loss

```

Training on dummy paired data. Real image-caption training is Task 5+. For now, demonstrate the model works mechanically: create a *synthetic* paired dataset. For example, pair each CIFAR-10 image with the text "this is a [class name]" — 10 distinct templates. Train the model to predict the caption given the image. Loss should drop substantially over a few thousand steps as the model learns to associate image classes with their text labels.

Expected outcome: loss drops from $\sim \log(\text{vocab_size})$ to ~ 0.5 or lower. When you sample from the model conditioned on an image, it should generate the correct caption template most of the time. This is a tiny, contrived task — the point is not the result, it is to verify that gradients flow through both the vision encoder and the text decoder via cross-attention, and that the model can learn cross-modal associations.

⚠ This is not real captioning yet

Synthetic captions like "this is a cat" are not real natural language. A model trained on this will not generalize to a real captioning dataset. The purpose of Part C is engineering verification: do the gradients flow correctly through the multimodal stack? Does the cross-attention learn anything? That's it. Tasks 5–6 will replace this with real Flickr8k captions and contrastive loss.

🔍 CURIOSITY CORNER — Two design philosophies — fusion strategies

The cross-attention approach you just built is sometimes called "late fusion via cross-attention" or "adapter-style fusion". Vision and language are processed separately, then mixed at specific layers.

Alternative: "early fusion". Concatenate image patch tokens and text tokens into one long sequence and do regular self-attention over both. This is what models like ViLT and LLaVA's projection-based approach do — simpler, but the self-attention has to learn cross-modal routing from scratch.

Cross-attention (what you built) is used by Flamingo, BLIP-2 (Q-Former), and historically by image captioning models. Early fusion is more common today for chat-style LLMs with vision. Each has trade-offs; reading the architecture sections of these papers is illuminating once you've built one yourself.

Part D — Visualize Cross-Attention

Time: about 2 hours.

After training, extract the cross-attention weights from one of your decoder blocks. For a given image and the generated caption, plot a heatmap showing which image patches each generated word attended to. If the model learned anything useful, you might see (for example) the model attending more to central patches when generating the class word.

Save as `task4/attention_viz.png`. Discuss in your writeup what you see — even "nothing meaningful" is a valid observation, since the task and dataset are tiny.

Part E — Conceptual Writeup

Time: 1–2 hours.

Answer in `task4/writeup.md`:

17. State the exact difference between self-attention and cross-attention in one or two sentences. What changes mechanically, and what changes semantically?
18. In your decoder block, you have three sub-layers: causal self-attention, cross-attention (not causal), and MLP. Explain why each one is or is not causally masked. What would go wrong if you swapped which were causal?
19. In cross-attention, the output sequence length equals the query length, not the context length. Why? Walk through the shapes.
20. If your vision encoder produced features with a different `n_embd` than your text decoder, how would you handle it? List two ways.
21. Describe what you saw in your cross-attention visualization. Did the model attend to anything meaningful? Why or why not, given the simplicity of your synthetic dataset?
22. Connect what you built in Task 4 to one real multimodal model you've heard of (CLIP, Flamingo, BLIP, LLaVA, GPT-4V — any). In your own words, what is the same and what is different?

Deliverables for Task 4

- `task4/cross_attention_toy.ipynb`
- `task4/multimodal.py`
- `task4/training_curve.png`
- `task4/attention_viz.png`
- `task4/samples.txt` — model outputs given a few test images.
- `task4/writeup.md`

What Comes After Task 4

At this point you have all the architectural pieces of a multimodal model:

- A vision encoder (ViT) that turns images into sequences of patch tokens.
- A text encoder/decoder that handles language.
- Cross-attention that lets them communicate.

What you don't yet have is a way to train them on real data toward a useful objective. That's Tasks 5–7:

- **Task 5 — Contrastive learning and InfoNCE loss.** Instead of next-token prediction, train two encoders (image and text) to produce embeddings such that matched image-text pairs are close and mismatched pairs are far apart. This is how CLIP works.
- **Task 6 — Training on Flickr8k.** Real images, real captions. Train end-to-end. Debug gradient flow at scale.
- **Task 7 — Inference and retrieval.** Text-to-image and image-to-text retrieval. Evaluation metrics like Recall@K.

Every one of those builds directly on what you have in Tasks 0–4. You're past the architectural hump. From here on the focus shifts from "building the model" to "training and using the model."

If Task 3 felt like the moment where the architecture clicked, and Task 4 felt like the moment where the two halves connected, you're exactly where you should be. Keep going.